Vibecoding Guiado: Desarrollo de Mini-Juego 2D con IA

Informe de Desarrollo, Ingeniería de Prompts y Validación

Proyecto:	Cosmic Runner 2D - Cyber Escape (HTML5 Canvas & JS)	**Metodología:**	Vibecoding & Prompt Engineering Iterativo

1. DESCRIPCIÓN GENERAL DEL PROYECTO

Se desarrolló un videojuego arcade 2D de tipo *infinite runner* denominado **"Cosmic Runner"**. El juego hace uso de gráficos vectoriales procedimentales en HTML5 Canvas, física de movimiento y gravedad para el personaje (astronauta), un sistema de partículas dinámicas para la propulsión y explosiones, efectos de sonido sintetizados mediante la Web Audio API y controles adaptativos para PC y móviles.

2. EVIDENCIA DE VIBECODING (PROMPTS UTILIZADOS)

Etapa / Prompt	Lo que se le pidió a la IA	Razón de ajuste / Refinamiento
1. Estructura Base	"Crea un mini-juego arcade 2D en HTML5 Canvas y JS nativo. El jugador debe ser un astronauta dibuajdo proceduralmente que esquiva obstáculos con gravedad y saltos."	La física inicial era muy rígida. Se ajustó para suavizar la aceleración de la gravedad y permitir un control continuo de propulsión.
2. Sprites y Efectos Visuales	"Añade un sistema de partículas para el jetpack y genera orbes coleccionables con brillo intermitente para incrementar la puntuación."	Se refinó el manejo de memoria en Canvas para evitar la acumulación excesiva de partículas que reducía los FPS.
3. Mecánicas Avanzadas	"Integra 3 power-ups (Escudo, Imán y Puntos 2X), sonidos con Web Audio API y botones táctiles adaptativos para pantalla de dispositivos móviles."	Se implementaron eventos táctiles (touch events) para garantizar la jugabilidad multiplataforma sin depender de librerías externas.

3. ITERACIÓN Y MEJORA SOLICITADA

Mejora clave implementada: Adición de fondo con desplazamiento de paralaje (*parallax scrolling*) y ajuste en las cajas de colisión (*hitboxes*).
Propósito: Otorgar profundidad tridimensional al escenario espacial y garantizar que la detección de impactos entre el jugador, los meteoritos y los orbes sea precisa y justa.

4. VALIDACIÓN Y CORRECCIÓN DE ERRORES

- **Método de prueba:** Pruebas de caja negra probando eventos de teclado (Flechas/Espacio) y respuestas táctiles en móviles, además de stress-testing con la generación constante de entidades.
- **Error identificado y solucionado:** Inicialmente, tras jugar varias partidas continuas, la acumulación masiva de partículas del propulsor producía un ligero retardo (*lag*). Se solucionó agregando un límite máximo de vida útil a las partículas y eliminándolas del arreglo en memoria al expirarse (`p.life <= 0`).

5. REFLEXIÓN FINAL

- **Aprendizaje con IA:** Se comprendió la arquitectura de desarrollo en ciclo cerrado (`requestAnimationFrame`), la gestión de estados (Start, Playing, GameOver) y el dibujo procedimental en Canvas.
- **Ventajas del Vibecoding:** Permite construir prototipos altamente funcionales y completos con audio y efectos avanzados en una fracción del tiempo habitual.
- **Límites identificados:** Exige que el programador comprenda la optimización de código y la gestión de recursos para evitar caídas de rendimiento o comportamientos inesperados.

6. CÓDIGO FUENTE GENERADO (RESUMEN ESTRUCTURAL DEL MINI-JUEGO)

A continuación se presenta la estructura esencial del motor 2D desarrollado en un único archivo autocontenido:

```
<!DOCTYPE html> <html lang="es"> <head> <meta charset="UTF-8"> <title>Cosmic Runner 2D</title
> <script src="https://cdn.tailwindcss.com"></script> </head> <body class="flex flex-col items-cent
er justify-center min-h-screen bg-slate-950 text-white"> <canvas id="gameCanvas" width="960" height
="540"></canvas> <script> // --- SINTETIZADOR DE AUDIO (Web Audio API) --- class SoundFX { construc
tor() { this.ctx = new (window.AudioContext || window.webkitAudioContext)(); } jump() { /* Oscilado
r para propulsión Jetpack */ } coin() { /* Tono agudo para orbes */ } powerup() { /* Arpegio ascend
ente */ } explosion() { /* Onda diente de sierra descendente */ } } // --- JUGADOR Y SPRITES PROCED
URALES --- const player = { x: 100, y: 300, w: 40, h: 55, vy: 0, gravity: 0.4, thrust: -0.85, shiel
d: false, magnet: false, doublePts: false, update() { if (keys.up) this.vy += this.thrust; this.vy
+= this.gravity; this.y += this.vy; }, draw() { // Dibujo del personaje (Traje, Visor, Jetpack) con
HTML5 Canvas API } }; // --- SISTEMA DE ENTIDADES Y COLISIONES --- class Obstacle { /* Meteoritos y
Drones Malvados */ } class Item { /* Orbes de puntos y Power-Ups */ } class Particle { /* Partícula
s de fuego y destellos */ } // --- BUCLE PRINCIPAL (Game Loop) --- function gameLoop() { ctx.clearR
ect(0, 0, 960, 540); drawBackground(); // Parallax scrolling if (gameState === 'PLAYING') { player.
update(); player.draw(); spawnManager(); // Spawning aleatorio checkCollisions(); } requestAnimatio
nFrame(gameLoop); } </script> </body> </html>
```